ATHENS UNIVERSITY OF ECONOMICS AND BUSINESS
DEPARTMENT OF INFORMATICS
Computer Systems Organization
MIPS Assembly Assignment
ATHANASIOS GOURDOMICHALIS

Introduction

In the **MIPS32** processor, dynamic memory allocation is performed using system call 9, which returns a pointer to the memory region allocated by the system. The size, in bytes, of the allocated memory region is passed as a value in register **$a0**.

sbrk - Memory Allocation:

- system call 9,
- **$a0** = number of bytes of storage desired,
- **$v0** = address of block

For example, the following program reads **two integers** and stores them in memory within a region **allocated dynamically** during program execution, **without** declaring or storing them in an integer array in the program's data segment. Finally, it displays the two integers.

```
.text

.globl main

main:
li   $a0, 8          # 2 x sizeof(int)
li   $v0, 9          # sbrk - system call 9, call service to
syscall              # allocate 2 words of memory (8 bytes)

move $t1, $v0        # $t1 = address of the first word in memory

li   $v0, 5          # read (int1)
syscall
sw   $v0, ($t1)      # store int1 in the first position

li   $v0, 5          # read (int2)
syscall
sw   $v0, 4($t1)     # store int2 in the second position

lw   $a0, ($t1)      # print (int1)
li   $v0, 1
syscall

lw   $a0, 4($t1)     # print (int2)
li   $v0, 1
syscall

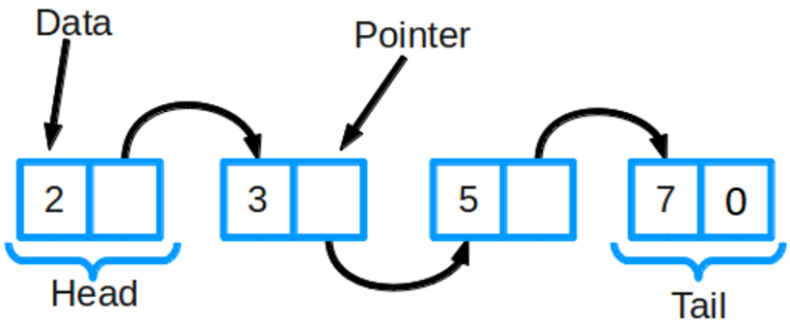
li   $v0, 10         # exit
syscall
```

Assignment Description

In the case of a **singly linked list**, each node can be a memory region allocated in the manner described above, consisting of two integer fields:

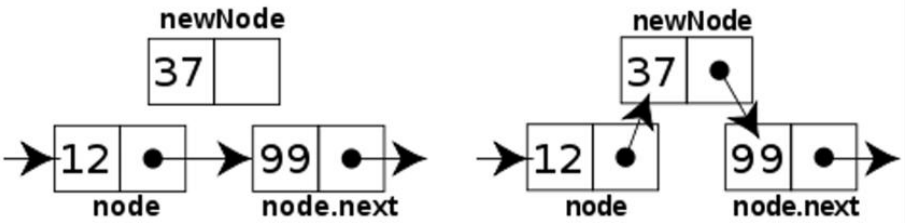
- The first integer represents the **data** of the node.
- The second integer represents the **link** (pointer) to the next node.

The last node in the list can have a **link value of zero** (NULL).

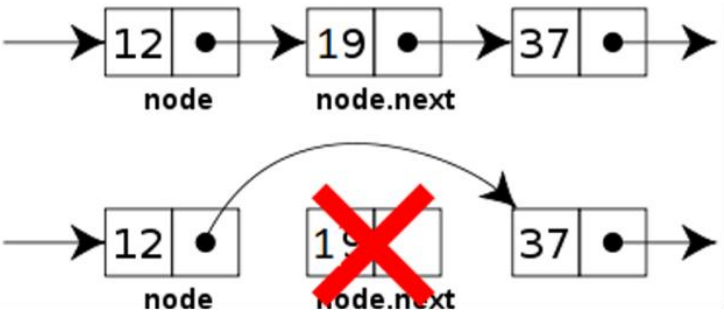


Write a program in **MIPS32** assembly language that creates and manages a dynamic linked list of integers. Each integer must be stored in a node of the list. Node insertion and deletion must be implemented in such a way that the list remains sorted in ascending order at all times:

Node Insertion into the list:



Node Deletion from the list:



The user interface must be implemented using a menu that provides the user with the following options:

1. **Insert a node into the list** in its correct position, ensuring that the list remains sorted in ascending order after the insertion. It first reads the integer to be inserted from the user and then calls the InsertNode subroutine to insert the integer into the list as a new node.
2. **Delete a node from the list**, ensuring that the list remains sorted in ascending order. If the list is not empty, it reads the integer to be deleted from the user and calls the DeleteNode subroutine to delete the node containing that integer from the list. Otherwise, it displays a message indicating that the list is empty.
3. **Display the list in ascending order** (view data from the first node to the last node). If the list is not empty, it calls the PrintList subroutine, which displays the list of integers. Otherwise, it displays a message indicating that the list is empty.

(Detailed descriptions of the subroutines are provided below).

Implementation Remarks:

- **Dynamic Allocation Requirement:** The implementation must be carried out as described above using **dynamic memory allocation** system calls (sbrk / syscall 9) and **not**, for example, by using an integer array declared in the .data segment.
- **Modular Structure via Procedures:** All menu operations must be implemented through (by invoking) **subroutines** (procedures) that you will develop as follows:

InsertNode Subroutine

- **Arguments:** The integer to be inserted into the list, and the address of the head (first node) of the list.
- **Function:** Calls the **CreateNode** subroutine to allocate a new node and inserts/links the new node into the list at the correct position (ensuring the list remains sorted).
- **Return Value:** The address of the head (first node) of the list.

CreateNode Subroutine

- **Argument**: The integer to be contained within the new node.
- **Function**: Creates a new node containing the integer passed as an argument.
- **Return Value**: The address of the newly created node.

DeleteNode Subroutine

- **Arguments:** The integer to be deleted from the list, and the address of the head (first node) of the list.

  Function: Deletes the node containing the target integer from the list. If the integer occurs multiple times in the list, it deletes a single occurrence. The list remains sorted after the deletion. If the integer is not found in the list, an appropriate message is displayed to the user.
- **Return Value:** The address of the head (first node) of the list.

PrintList Subroutine

- **Argument:** The address of the head (first node) of the list.
- **Function:** Displays the sorted list of integers.

Instructions:

- **Calling Sequence:** Wherever a subroutine is called, include a comment specifying what the **calling sequence** consists of and which instructions implement it.
- **Algorithm Description:** Before the body of each subroutine, describe its **algorithm** in pseudocode within a comment block.
- **Prologue & Epilogue**: Within each subroutine, clearly annotate with comments which instructions constitute its **prologue and epilogue**, explaining their purpose.

- **In-Line Comments:** Write concise inline comments for individual instructions or groups of instructions (for example):

```
printQuestion:    li $v0, 4          # print(quest)
                  la $a0, quest
                  syscall
```

- **Register Preservation:** During implementation, properly preserve register values on the stack (using register save conventions) wherever required.
- **Simulator Execution:** Execute your program in the SPIM simulator and ensure that it is completely free of syntax or runtime errors.
- **Sample Execution:** On the following page, you can view an example of program execution.

Program execution example

PAGE 1

```
----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
2
The list is empty


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
3
The list is empty
```

PAGE 2

```
----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
1
Enter a value
30
Value added!


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
1
Enter a value
10
Value added!

----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
1
Enter a value
20
Value added!


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
1
Enter a value
20
Value added!


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
3
10 20 20 30
```

PAGE 3

```
----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
2
Delete a value
25
The value you entered wasn't found


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
2
Delete a value
20
Value deleted!


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
3
10 20 30

----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
2
Delete a value
10
Value deleted!


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
3
20 30
```

PAGE 4

```
----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
2
Delete a value
20
Value deleted!


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
2
Delete a value
30
Value deleted!


----------------------
(1) Insert data!
(2) Delete data!
(3) Show list ascending!
(0) Exit Program!
----------------------
Please choose an option:
3
The list is empty
```